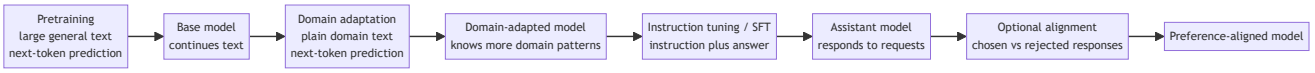
Non-Instruction LLM Fine-Tuning: Notebook and Study Guide

This guide explains the concepts in the fine-tuning notes and walks through the code in non_Instruction_pretrain_llm_finetuning_on_domain_specific_data.ipynb. It also calls out practical problems in the current notebook and gives safer code patterns. The notebook has not been executed, so the examples below are explanations and suggested corrections, not verified training results.

1. The Training Pipeline

A language model is usually adapted in stages. Each stage has a different goal and a different kind of training data.



- **Pretraining** teaches general language patterns by predicting the next token in huge text collections. It is usually called self-supervised because the text itself supplies the prediction target.
- **Domain adaptation** (also called continued pretraining or non-instructional fine-tuning) uses plain text from a target domain. Its objective is still next-token prediction. It does not, by itself, guarantee that the model will answer questions.
- **Instruction tuning** trains on prompt/response examples so the model learns to respond to tasks and requests. It is supervised fine-tuning (SFT).
- **Preference alignment** uses comparisons between responses to teach the model which response is preferred. It is optional and comes after a useful instruction-tuned model in many workflows.

Two independent choices are often confused:

Choice	Question	Examples
Training data / behavior	What should the model learn to do?	Plain text, instruction/answer pairs, response preferences
Parameter method	Which weights should change?	Full fine-tuning, frozen layers, LoRA/QLoRA

For example, LoRA can be used with plain-text domain adaptation or with instruction tuning. LoRA describes *how* to update the model; the data format describes *what behavior* to train.

2. Terms in the Notes

Pretraining and next-token prediction

A tokenizer converts text into token IDs. The model sees a sequence and learns to predict the next token at each position. For example:

Context given to model	Expected next token
`The medicine should be stored`	`below`
`The medicine should be stored below`	`25`

The model's loss is calculated between predicted tokens and the true next tokens. Causal language models shift predictions internally, so labels commonly contain the same token IDs as inputs.

A **base model** can produce fluent continuations without knowing that a user expects a direct answer. “It knows facts” is only a shorthand: a model stores statistical patterns, can be wrong, and does not guarantee retrieval of a fact on demand.

Full and partial fine-tuning

- **Full fine-tuning:** Update essentially all model parameters. It is flexible but needs memory for weights, gradients, optimizer state, and activations. Memory depends on precision, optimizer, sequence length, and implementation; estimates such as 16

bytes per parameter are rough, not fixed laws.

- **Layer freezing / partial fine-tuning:** Freeze some parameters and train selected layers or a new output layer. This reduces trainable parameters but requires knowing the model's layer names and structure.
- **PEFT:** Parameter-efficient fine-tuning. Keep the base mostly frozen and train a small adapter or selected parameter set.

PEFT methods named in the notes

1. **LoRA:** Add trainable low-rank matrices to selected linear layers. The adapted weight is often described as $W' = W + BA$ (matrix order depends on convention). Only the small matrices are trained.
2. **QLoRA:** LoRA training with the frozen base model loaded in 4-bit quantization. Quantization reduces base-weight memory; it does not mean the adapter is 4-bit. The notebook uses 8-bit loading, so that particular code is quantized LoRA, not the usual 4-bit QLoRA setup.
3. **DoRA:** Separates weight magnitude and direction and adapts them in a LoRA-like way. It may help some tasks, but is not guaranteed to outperform LoRA.
4. **Adapter layers:** Small trainable bottleneck modules inserted into transformer blocks. They add parameters and can add inference latency.
5. **BitFit:** Trains bias terms while freezing most other weights.
6. **IA3:** Learns scaling vectors applied to selected internal activations, such as attention and feed-forward activations.
7. **Prefix tuning:** Learns vectors used as extra attention context at each layer, typically in keys and values.
8. **Prompt tuning:** Learns soft prompt embeddings at the model input. These are vectors, not literal words typed by a user.

The notes group LoRA and QLoRA into one item, which is why their list has seven entries. They are related, but QLoRA specifically adds 4-bit base-model quantization.

Non-instructional versus instruction data

Type	One training row looks like	Main learning goal	Typical output
Non-instructional / domain adaptation	A paragraph from a manual or report	Learn domain vocabulary, style, and token patterns	Text continuation
Instruction tuning / IFT	A request plus a suitable response	Follow tasks and answer in the desired format	Direct response
Preference alignment	A prompt, chosen answer, rejected answer	Prefer one response over another	More preferred response behavior

Example plain-text row: `Product X must be stored below 25 C.`

Example instruction row:

```
Instruction: What is the storage condition for Product X?
Response: Store Product X below 25 C.
```

The guide's sequence “domain tune, then instruction tune, then optional preference alignment” is a reasonable recipe, not a requirement. Retrieval-augmented generation (RAG) may be better than trying to bake changing or safety-critical facts into model weights.

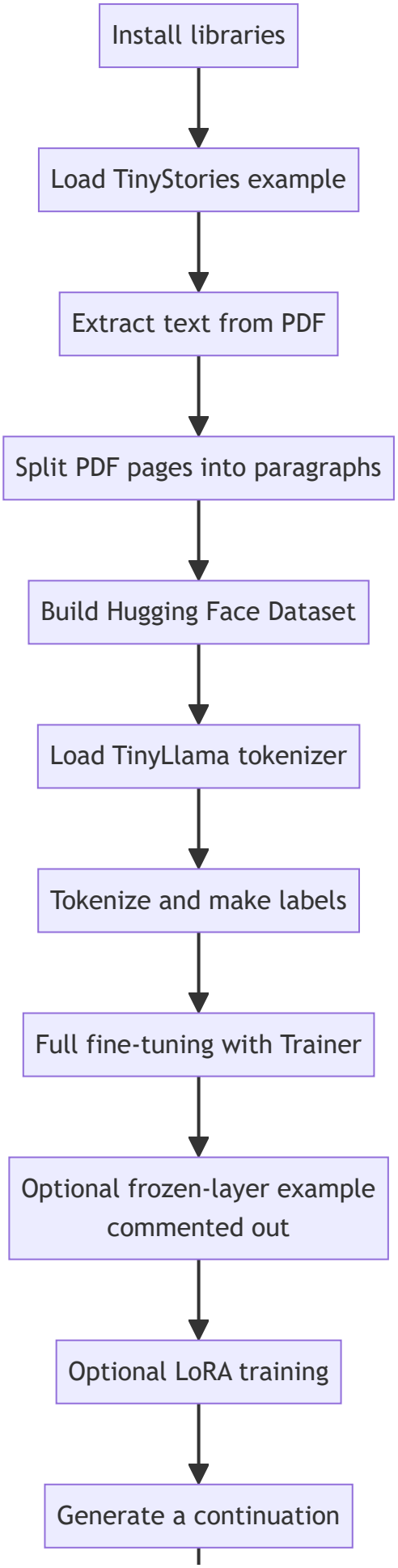
Preference alignment

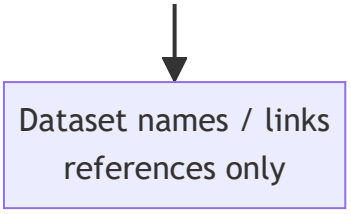
A preference example usually has the form `(prompt, chosen response, rejected response)`. The labels are comparisons, not absolute truth labels.

- **RLHF:** Reinforcement Learning from Human Feedback. A common recipe starts from an SFT model, trains a reward model from preference comparisons, then uses a method such as PPO to optimize the language model while constraining drift from the SFT model.
- **DPO:** Direct Preference Optimization. Optimizes from chosen/rejected examples directly, without a separately trained reward model and PPO stage in the common recipe. It is often simpler, but still depends on preference data quality.
- **RLAIF:** Reinforcement Learning from AI Feedback. An AI judge supplies preference labels instead of, or in addition to, human labels.

These are broad descriptions. Alignment has more approaches than these three, and RLHF does not always mean precisely one fixed PPO implementation.

3. Notebook at a Glance


Install libraries
Load TinyStories example
Extract text from PDF
Split PDF pages into paragraphs
Build Hugging Face Dataset
Load TinyLlama tokenizer
Tokenize and make labels
Full fine-tuning with Trainer
Optional frozen-layer example commented out
Optional LoRA training
Generate a continuation



The notebook first loads TinyStories, but later replaces the variable `dataset` with a dataset made from the extracted PDF paragraphs. Therefore the main training cells use the custom PDF data if run top-to-bottom. The final dataset-name cells only list possible sources; they do not load them into the training pipeline. Likewise, the notebook installs `trl`, but does not implement RLHF or DPO training.

4. Notebook Code Walkthrough

A. Install libraries

The notebook installs `peft`, `bitsandbytes`, `transformers`, `accelerate`, `trl`, and `PyMuPDF` using notebook shell commands.

- `transformers`: model/tokenizer loading and `Trainer`.
- `datasets`: loading and mapping datasets. It is used but is not explicitly installed in the notebook's initial install cells; it may already be present in hosted environments.
- `peft`: LoRA and other PEFT methods.
- `bitsandbytes`: low-bit model loading/training support on supported hardware/software setups.
- `accelerate`: device placement and distributed-training utilities used by Transformers.
- `trl`: preference and reinforcement-learning training utilities; installed but unused in this notebook.
- `PyMuPDF` provides the `fitz` import used for PDF extraction.

In a notebook, `%pip install ...` is usually preferable because it installs into the active kernel environment. After changing core packages, restart the kernel if imports behave inconsistently. Compatibility depends on Python, PyTorch, CUDA, operating system, and library versions. Bitsandbytes quantization is especially hardware/platform dependent.

B. Load a built-in dataset

```
from datasets import Dataset, load_dataset

dataset = load_dataset("roneneldan/TinyStories", split="train")
print(dataset)
print(dataset[0])
```

- `load_dataset` downloads or opens a dataset from Hugging Face Datasets.
- `split="train"` selects only the training split.
- `dataset[0]` inspects one row.

TinyStories is a general story dataset, useful for experimenting with the API but not a pharmaceutical domain dataset. The PDF-derived dataset later overwrites `dataset`.

C. Extract text from a PDF

```
import fitz

def extract_text_from_pdf(pdf_path):
    text_blocks = []
    with fitz.open(pdf_path) as doc:
        for page in doc:
            text = page.get_text("text").strip()
            if text:
                text_blocks.append(text)
```

```
    return text_blocks

pdf_texts = extract_text_from_pdf("/content/Metformin.pdf")
```

`fitz.open` opens the PDF; each page's embedded text is extracted and blank pages are skipped. `/content/Metformin.pdf` is a Google Colab-style path, not a Windows path. Change it to the actual local file path or upload/mount the PDF where the notebook runs.

This works when the PDF contains a text layer. If it is a scan/image-only PDF, `get_text()` may return empty text. Use OCR first, inspect the result, and correct OCR errors before training. The newly attached six-page PDF is reported as not text-parseable, so it likely needs OCR or manual transcription for programmatic extraction.

D. Split and package the text

```
import re

def split_paragraphs(pages):
    paragraphs = []
    for page_text in pages:
        chunks = re.split(r"\n\s*\n", page_text)
        for chunk in chunks:
            clean = chunk.strip()
            if len(clean) > 30:
                paragraphs.append(clean)
    return paragraphs

paragraphs = split_paragraphs(pdf_texts)
data = [{"text": paragraph} for paragraph in paragraphs]
dataset = Dataset.from_list(data)
```

- The regex splits at blank-line boundaries.
- `strip()` removes whitespace around each chunk.
- The 30-character filter discards short chunks; it is a crude heuristic and can accidentally remove useful headings or short definitions.
- `Dataset.from_list` creates a Hugging Face dataset with a `text` column.

Important limitation: PDF extraction often produces one newline per visual line, not blank lines between paragraphs. In that case this regex may leave a whole page as one chunk. Also, the later tokenizer truncates each row to 512 tokens, silently throwing away the rest of a long row. Clean headers/page numbers and split into token-length chunks so all useful text is included. Keep paragraphs from the same source document together when creating train/validation splits to reduce leakage.

E. Select model and tokenizer

```
model_name = "TinyLlama/TinyLlama-1.1B-intermediate-step-1431k-3T"

from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    Trainer,
    TrainingArguments,
    DataCollatorForLanguageModeling,
)

tokenizer = AutoTokenizer.from_pretrained(model_name)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token
```

`AutoTokenizer` selects the tokenizer configuration that belongs to the checkpoint. `AutoModelForCausalLM` selects a causal next-token prediction model. Use the same model checkpoint for tokenizer and model. The notebook's comments include Llama 2 and Llama 3.1 alternatives; gated models may require accepting their license and authenticating with Hugging Face.

If there is no dedicated padding token, the code aliases the end-of-sequence token as padding. That is common, but it makes it especially important to mask padded positions out of the loss.

The notebook describes this TinyLlama checkpoint as an intermediate checkpoint. Treat that as a checkpoint-specific detail and check its model card before relying on claims about its training stage or quality.

F. Tokenize and create labels

The notebook uses:

```
def tokenize_fn(examples):
    tokens = tokenizer(
        examples["text"],
        truncation=True,
        padding="max_length",
        max_length=512,
    )
    tokens["labels"] = tokens["input_ids"].copy()
    return tokens

tokenized = dataset.map(
    tokenize_fn,
    batched=True,
    remove_columns=["text"],
)
```

- `batched=True` passes a batch of strings to the tokenizer, which is faster than one row per call.
- `truncation=True` cuts any row longer than 512 tokens.
- `padding="max_length"` pads every row to 512 tokens, creating fixed-size examples.
- `input_ids` are token numbers; `attention_mask` is 1 for real tokens and 0 for padding.
- `labels` are the expected token IDs used to calculate the causal language-modeling loss.
- `remove_columns=["text"]` drops the original text after tokenization.

`max_length=512` is the training example length selected here. It is not a universal standard or necessarily the model's maximum context length. A tokenizer's token count is not the same as a word count.

Bug to fix before training: the current code copies padded token IDs into `labels`, so the loss may train the model to predict padding. Mask labels wherever `attention_mask` is 0. Also use a validation set and inspect sample tokenized rows before launching training.

A simple safe label-masking pattern is:

```
def tokenize_fn(examples):
    tokens = tokenizer(
        examples["text"],
        truncation=True,
        padding="max_length",
        max_length=512,
    )
    tokens["labels"] = [
        [token_id if mask else -100 for token_id, mask in zip(ids, attention)]
        for ids, attention in zip(tokens["input_ids"], tokens["attention_mask"])
    ]
    return tokens
```

Hugging Face loss functions ignore label value `-100`. Another conventional option is to let `DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)` build labels and mask padding; do not mix strategies without checking what labels the collator produces.

G. Full fine-tuning with `Trainer`

```
model = AutoModelForCausalLM.from_pretrained(model_name)

training_args = TrainingArguments(
    output_dir="./llama-pharma-domain",
    overwrite_output_dir=True,
    num_train_epochs=2,
    per_device_train_batch_size=2,
    save_steps=500,
    save_total_limit=2,
    logging_steps=50,
    learning_rate=2e-5,
    fp16=True,
    report_to="none",
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized,
)
trainer.train()
```

There are no PEFT adapters and no parameters are frozen in this code, so it is **full fine-tuning**. `Trainer` computes the loss, backpropagates, updates trainable weights, logs progress, and saves checkpoints according to the arguments.

Argument	Meaning
`output_dir`	Where checkpoints and outputs are saved
`overwrite_output_dir`	Permit reuse/overwrite of the output directory
`num_train_epochs`	Number of passes through the training data
`per_device_train_batch_size`	Examples per device for one forward/backward pass
`save_steps`	Save checkpoint every this many optimizer steps
`save_total_limit`	Retain only this many checkpoints
`logging_steps`	Emit training logs this often
`learning_rate`	Size of optimizer updates; needs tuning
`fp16`	Request half-precision training; requires compatible hardware
`report_to="none"`	Disable external experiment reporting

A batch size of 2 means two examples per device per micro-batch. If using gradient accumulation or multiple devices, the effective batch size changes. `fp16=True` can fail on unsupported devices; use a compatible precision or disable it. The notebook's tiny demo dataset has only four PDF paragraphs, so two epochs are not a meaningful domain training run and are likely to overfit. Add enough clean data, a held-out validation set, and baseline evaluation.

The `help(TrainingArguments)` cell is for interactive API browsing; it is not part of the training logic. The `DataCollatorForLanguageModeling` import in the first path is not actually passed to `Trainer`.

H. Frozen-layer example

The long code cell beginning with `#` is entirely commented out, so it does not execute. It demonstrates the older partial fine-tuning idea: freeze all parameters, then unfreeze a few transformer layers and the language-model head.

```
for parameter in model.parameters():
    parameter.requires_grad = False

for name, parameter in model.named_parameters():
    if any(f"layers.{index}." in name for index in range(20, 24)):
        parameter.requires_grad = True
    if "lm_head" in name:
        parameter.requires_grad = True
```

Do not copy the layer numbers blindly. They depend on the architecture and layer count. TinyLlama has fewer than 24 transformer blocks, so indices 22 and 23 in this example do not identify real blocks; verify `model.named_parameters()` and the model config first. Also verify the actual parameter naming convention (`model.layers` , `transformer.h` , etc.).

I. LoRA training path

The notebook does this in outline:

1. Collect garbage and clear unused CUDA cache.
2. Set `model` to a model-name string and load its tokenizer.
3. Tokenize the current `dataset` .
4. Load the base model with `load_in_8bit=True` .
5. Create `LoraConfig` , attach it with `get_peft_model` , then train.

```
lora_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=8,
    lora_alpha=16,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
    bias="none",
)
```

- `task_type=CAUSAL_LM` tells PEFT this is next-token language modeling.
- `r` is the low rank / size of the adapter matrices.
- `lora_alpha` scales the LoRA update.
- `target_modules` selects attention projections to adapt; names must exist in the selected model.
- `lora_dropout` regularizes adapter inputs during training.
- `bias="none"` leaves base bias parameters untouched.

`get_peft_model` wraps the base model with the adapter, generally freezing the base and making adapter parameters trainable. `gc.collect()` and `torch.cuda.empty_cache()` can release unused objects/cache, but do not free memory held by live Python objects or make an oversized model fit by themselves.

The notebook loads in **8-bit**, not 4-bit. To call it QLoRA, use 4-bit configuration; otherwise call it 8-bit LoRA. Low-bit training commonly also calls `prepare_model_for_kbit_training` . A clearer variable name avoids overwriting the model-name string with the model object. Suggested pattern:

```
import torch
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
from peft import LoraConfig, TaskType, get_peft_model, prepare_model_for_kbit_training

model_name = "TinyLlama/TinyLlama-1.1B-intermediate-step-1431k-3T"
```

```
quantization = BitsAndBytesConfig(load_in_8bit=True)

base_model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=quantization,
    device_map="auto",
)
base_model = prepare_model_for_kbit_training(base_model)

lora_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=8,
    lora_alpha=16,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
    bias="none",
)
model = get_peft_model(base_model, lora_config)
model.print_trainable_parameters()
```

For QLoRA, replace the 8-bit configuration with a tested 4-bit `BitsAndBytesConfig` (commonly NF4, with an appropriate compute dtype), and confirm hardware/library compatibility. Quantization settings and device placement are environment-specific.

The LoRA `TrainingArguments` use one example per device and `gradient_accumulation_steps=8`. On one device, this gives an effective batch of about 8 examples per optimizer update. The listed learning rate `2e-4` is a starting value, not a guaranteed good value. Add the same padding-label fix and validation split from the full fine-tuning path.

J. Save, reload, and generate

The notebook assumes this path:

```
model_path = "/content/tinyllama-lora/checkpoint-5"
```

That path is fragile: checkpoint intervals default to a different cadence unless configured, and the number of optimizer steps depends on the dataset and batch settings. With four examples, five epochs, and accumulation of eight, there may be only a few optimizer updates, so `checkpoint-5` may never exist. Save explicitly to a known final path.

PEFT checkpoints usually contain adapter weights, not a complete standalone base model. Reload the same base checkpoint, then attach the adapter:

```
from peft import PeftModel

base_model = AutoModelForCausalLM.from_pretrained(
    model_name,
    device_map="auto",
)
model = PeftModel.from_pretrained(base_model, "./tinyllama-lora-final")
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

To save the adapter after training, use `trainer.save_model("./tinyllama-lora-final")` and keep the base model name/config recorded. Depending on Transformers/PEFT versions, also save the tokenizer with `tokenizer.save_pretrained(...)`.

The generation code is:

```
prompt = "Clinical trials demonstrated that combining Atorvastatin with Ezetimibe"
inputs = tokenizer(prompt, return_tensors="pt").to("cuda")
outputs = model.generate(
    **inputs,
```

```
    max_new_tokens=100,
    temperature=0.8,
    top_p=0.9,
    do_sample=True,
    repetition_penalty=1.1,
)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

- `max_new_tokens` limits generated tokens, not total input-plus-output length.
- With `do_sample=True`, `temperature` controls randomness and `top_p` uses nucleus sampling.
- `repetition_penalty` discourages repeated text; it can also distort useful repetition.
- The output includes the prompt because the entire sequence is decoded.
- `.to("cuda")` assumes one CUDA device. It may fail on CPU/MPS or when model sharding requires a different input device. Match input placement to the model's actual device setup.

Generation from a plain-text-trained model is text continuation, not evidence that it has learned reliable question answering.

K. Dataset catalogs at the end

The notebook lists candidate datasets for plain text, instruction tuning, and preferences. These are references, not training code. Before use, check current dataset IDs, schema, license, privacy terms, split names, quality, and intended use. Dataset availability and names can change.

The URL-only code cell contents are not valid Python statements if executed. Move these URLs to Markdown or write them as strings/comments. The notebook does not currently load an instruction dataset, format chat examples, run SFT on instruction/answer data, load preference pairs, or train with RLHF/DPO. `trl` being installed does not implement those stages automatically.

5. PDF / Image Notes Explained

The attached handwritten diagram groups fine-tuning this way:

- **Parameter level:** full fine-tuning versus partial fine-tuning. Full updates all weights and needs substantially more memory. Partial updates some weights or uses PEFT.
- **Old-school partial tuning:** freeze all but an output layer, or freeze early blocks and train later blocks.
- **PEFT list:** LoRA/QLoRA, DoRA, adapters, BitFit, IA3, prefix tuning, and prompt tuning.
- **Data level:** non-instructional plain text versus instruction-plus-response data.
- **Alignment:** preference-based learning from human or AI comparisons. The diagram labels PPO as a reinforcement-learning algorithm used in a common RLHF recipe and DPO as direct preference optimization.
- **Pharmaceutical example:** start with a pretrained base model, adapt it with pharmaceutical plain text, instruction-tune it with domain questions and answers, and optionally preference-tune it. The diagram correctly distinguishes domain data from instructions.

The notebook currently demonstrates PDF text extraction, plain-text next-token training, and an LoRA adaptation path. It does **not** complete the instruction-tuning, DPO, or RLHF stages drawn in the notes.

6. Missing Steps Before a Real Run

1. **Check the environment:** choose Colab or local Windows, use a valid PDF path, confirm GPU/PyTorch/CUDA, and ensure package versions are compatible.
2. **Inspect the PDF text:** confirm extraction/OCR quality, remove page furniture and broken line wraps, and verify sensitive or incorrect facts manually.
3. **Build token-sized chunks:** do not drop long paragraphs at the 512-token truncation boundary; preserve document boundaries and useful context.
4. **Split train and validation data:** split by document or source before chunking where possible, to avoid near-duplicate leakage.
5. **Mask padding labels:** exclude padding from the loss, then inspect labels and a decoded example.
6. **Pick training mode deliberately:** the first `Trainer` path is full fine-tuning; the PEFT path is LoRA with 8-bit loading. Use 4-bit configuration if intending QLoRA.

7. **Track and evaluate:** compare the base and adapted models on held-out domain prompts; monitor loss, overfitting, generation quality, and factual accuracy.
8. **Save and reload correctly:** use an explicit output path and reload a PEFT adapter on its matching base model.
9. **Treat medical/pharmaceutical output as high risk:** fine-tuning is not a source of truth or a safety guarantee. Use citations/retrieval, expert review, privacy controls, and appropriate validation for consequential use.

7. Quick Self-Test

1. Why can a base model continue a paragraph instead of answering a question?
   Because pretraining optimizes next-token continuation, not necessarily instruction-following behavior.
2. What differs between domain adaptation and instruction tuning?
   Domain adaptation uses plain text; instruction tuning uses requests with target responses.
3. What do LoRA and QLoRA change?
   LoRA trains small low-rank adapters; QLoRA does that while loading the frozen base in 4-bit form.
4. Why mask padding in labels?
   Padding is structural filler, not a token the model should learn to predict as document content.
5. What is the difference between RLHF and DPO in the common recipes?
   RLHF commonly trains a reward model and uses RL such as PPO; DPO optimizes directly from preference pairs.
6. Does this notebook produce a chatbot?
   Not as written. It trains on plain text and generates a continuation. It needs instruction-formatted examples and SFT to demonstrate assistant behavior.